

ECOM EXPERTS

Optimice server y db

Manual de usuario, tecnico y de desarrollo

optimice server y db · 08/09/2026

Contenido

Qué hace este módulo

Instalación

Primera vez

Las 5 pestañas

 Dashboard

 Wizard

 Resultados

 Herramientas

 Configuración

Modo "automático" recomendado

Cosas que el módulo NO hace desde el navegador

Significado de los iconos / colores

Preguntas frecuentes

Soporte

1. Arquitectura general

2. Tablas BD

3. Hooks usados

4. Tabs admin

5. Flujo de datos

6. Seguridad

6.1 CSRF / Token

6.2 Whitelist de scripts

6.3 escapeshellarg / escapeshellcmd

6.4 Lectura de logs

6.5 pSQL en queries

7. Permisos archivo

8. Compatibilidad PHP

9. Multi-tienda

10. Performance

11. Backup / restauración

12. Logs de depuración

13. Cron / autoejecución

14. Errores comunes

1. Convenciones de código (CLAUDE.md - obligatorias)

2. Añadir un nuevo script al toolkit

3. Añadir una pestaña nueva

4. Añadir parseo de un nuevo tipo de log

5. Añadir un nuevo gráfico

6. Variables CSS personalizables

7. AJAX desde el frontend del módulo

8. Permisos de los .sh

9. Testing local

10. Estructura PSR (no aplicable)

11. Compatibilidad PHP 7.4 ↔ 8.x

12. Smarty templates

13. Linting

14. Versionado

Versión: 1.0.0 **Autor:** Ecom Experts ecomymseo@gmail.com

Qué hace este módulo

Audita y optimiza tu servidor y base de datos PrestaShop **paso a paso**, con un wizard guiado, gráficos y recomendaciones automáticas. Incluye un toolkit de 8 scripts shell que cubren desde el diagnóstico hasta el tuning de MariaDB.

Instalación

1. Sube la carpeta `ecom_serveraudit/` a `/modules/` de tu PrestaShop.
2. Backoffice → **Módulos** → **Gestor de módulos** → busca "Server Audit" → Instalar.
3. Una vez instalado, en el menú lateral izquierdo aparece **MEJORAS** → **Server Audit**.

Primera vez

1. Entra al módulo desde el menú **Server Audit**.
2. Pestaña **Wizard** → seguir los 5 pasos: - **Paso 1:** Diagnóstico (audit) — click "Ejecutar audit" - **Paso 2:** Snapshot en vivo (perf) — click "Ejecutar perf" - **Paso 3:** Slow queries 30s — click "Capturar 30s" - **Paso 4:** Aplicar correcciones — copia los comandos SSH - **Paso 5:** Tuning MariaDB — copia el comando SSH
3. Vuelve al **Dashboard** para ver el resultado con gráficos.
4. **Resultados** te muestra todos los logs generados para revisión.

Las 5 pestañas

Dashboard

Vista general con tarjetas KPI: - Tamaño de la BD y espacio reclamable - Tablas corruptas detectadas - Multi-tienda (cuántas tiendas) - Disco usado, memoria - Gráficos: top 10 tablas, datos > 2 años -

Recomendaciones automáticas según los problemas encontrados

Wizard

Asistente de 5 pasos con barra de progreso. Cada paso explica qué hace y te da: - Botón para ejecutar desde el navegador (cuando es seguro) - O bloque de comandos SSH para copiar/pegar (cuando requiere root)

Resultados

Lista todos los logs generados (audit, perf, slow, fix, indexes, tune, optimize) ordenados por fecha. Click "Ver" abre un visor con el contenido íntegro.

Herramientas

Tarjetas individuales por script. Lanza cada uno por separado.

Configuración

- **Modo depuración:** guarda logs internos del módulo
- **Aplicar permisos a los .sh:** chmod 700 para que sean ejecutables
- Información del sistema (PHP, MySQL, PrestaShop, shell_exec status)

Modo "automático" recomendado

Si quieres dejar el toolkit funcionando solo:

```
# Por SSH como root:
cd /home/<user>/web/<dominio>/public_html/modules/ecom_serveraudit/sh
sudo ./optimize.sh --init           # crea auditoria.conf
sudo nano auditoria.conf           # ajusta valores
sudo ./optimize.sh                 # dry-run
sudo ./optimize.sh --apply         # aplica

# Y en cron (lunes 4 AM):
sudo crontab -e
0 4 * * 1 /home/.../modules/ecom_serveraudit/sh/optimize.sh --apply --yes >/dev/null 2>&1
```

Cosas que el módulo **NO** hace desde el navegador

Por seguridad, las operaciones que requieren root **se ejecutan por SSH**:

Acción	Por qué	Solución
<code>fix.sh --apply</code> (modificar BD)	Modifica datos	Copia el comando SSH del wizard paso 4
<code>tune.sh</code> (reiniciar MariaDB)	Corta servicio	Copia el comando SSH del wizard paso 5
<code>indexes.sh --apply</code>	Modifica BD	Copia el comando SSH del wizard paso 4
<code>slow.sh -t 300</code> (>60s)	Ocupa recursos	Por SSH para no bloquear el navegador

Lo que SÍ se puede ejecutar desde el módulo: - `audit.sh --no-backup` (solo lectura, rápido) - `perf.sh` (solo lectura, instantáneo) - `slow.sh -t 30` (30s, activa slow log temporalmente)

Significado de los iconos / colores

-  Verde: estado saludable
-  Amarillo: warning, atención

-  Rojo: problema crítico, actuar
-  Azul: informativo

Preguntas frecuentes

¿Borra productos / clientes / pedidos? NO. Nunca. Los scripts shell tienen una whitelist explícita de tablas que pueden tocar (logs, cache, configuration, hooks, etc.).

¿Necesito tener algo instalado en el servidor? - Comandos básicos: `mysql / mariadb`, `mysqldump`, `gzip`, `find`, `awk`. - Recomendado: `mysqldumpslow` o `percona-toolkit` (para análisis slow log). - Opcional: `smartctl` (para diagnóstico SMART de discos).

¿Funciona con multi-tienda? Sí. El módulo detecta multi-tienda y los scripts respetan `id_shop` y `id_shop_group` en todas las operaciones.

¿Necesito acceso root? - Para audit/perf/slow desde el módulo: NO (usa el usuario web). - Para fix/tune/indexes con `--apply`: Sí, debes ejecutar por SSH como root.

¿Cómo recupero la BD si algo va mal? Los scripts hacen backup obligatorio antes de tocar la BD. Verás `.sql.gz` en `sh/` o `/auditoria/`. Restaurar:

```
gunzip -c <backup>.sql.gz | sudo mysql <db_name>
```

¿Y si veo "shell_exec deshabilitada"? Tu hosting tiene esa función bloqueada (típico hosting compartido). Solución: usa los comandos SSH que muestra el módulo, o pide a tu hosting habilitarla.

¿Cuándo debo lanzar optimize.sh? - Primera optimización completa de un cliente: una vez. - Mantenimiento periódico: semanal por cron. - Tras instalar/desinstalar muchos módulos: una vez. - Tras incidencias de lentitud: para diagnóstico.

¿Los gráficos no salen? Sustituye `views/js/chart.umd.min.js` por la build oficial de Chart.js 4.x:

```
curl -L https://cdn.jsdelivr.net/npm/chart.js@4/dist/chart.umd.min.js \
-o modules/ecom_serveraudit/views/js/chart.umd.min.js
```

Soporte

Ecom Experts · ecomseo@gmail.com

Ecom Server Audit - Documentación técnica

Versión: 1.0.0 **Autor:** Ecom Experts ecomymseo@gmail.com **Licencia:** AFL-3.0 **Compatibilidad:** PrestaShop 1.7 / 8.x / 9.x · PHP 7.4+

1. Arquitectura general

```
ecom_serveraudit/
├── ecom_serveraudit.php      (clase principal Module)
├── config.xml                (metadata módulo)
├── index.php                 (cabecera + redirect)
├── classes/
│   ├── Ecom_Serveraudit_Logger.php  (log interno con DEBUG flag)
│   ├── Ecom_Serveraudit_Runner.php  (ejecuta .sh y lista logs)
│   └── Ecom_Serveraudit_Parser.php  (parsea logs a array PHP)
├── controllers/admin/
│   ├── AdminEcomServerauditController.php      (panel admin con Smarty)
│   └── AdminEcomServerauditAjaxController.php  (endpoint JSON)
├── views/
│   ├── css/admin.css          (estilos)
│   ├── js/admin.js           (lógica frontend)
│   ├── js/chart.umd.min.js   (Chart.js - placeholder, sustituir)
│   └── templates/admin/
│       ├── main.tpl          (layout + tabs)
│       └── _partials/
│           ├── dashboard.tpl
│           ├── wizard.tpl
│           ├── results.tpl
│           ├── tools.tpl
│           └── settings.tpl
├── sh/                        (8 scripts shell del toolkit)
│   ├── audit.sh
│   ├── perf.sh
│   ├── slow.sh
│   ├── fix.sh
│   ├── indexes.sh
│   ├── tune.sh
│   ├── optimize.sh
│   └── INFO.txt
├── logs/                      (logs internos si DEBUG=1)
├── upgrade/                   (placeholder)
└── docs/                      (esta documentación)
```

2. Tablas BD

Este módulo **NO crea tablas** propias. Sólo guarda: - `ECOM_SERVERAUDIT_DEBUG` en `ps_configuration` (0/1) - `ECOM_SERVERAUDIT_AUTH_TOKEN` en `ps_configuration` (token interno) - `ECOM_SERVERAUDIT_LAST_RUN` en

`ps_configuration` (timestamp último run)

3. Hooks usados

- (Ninguno por defecto - el módulo es de **administración**, no de front)

4. Tabs admin

Crea el tab `AdminEcomServeraudit` colgado de `IMPROVE` (o `AdminParentModulesSf` como fallback) durante el `install()` y lo borra en `uninstall()`.

5. Flujo de datos



6. Seguridad

6.1 CSRF / Token

El AdminController de PrestaShop ya valida automáticamente el token CSRF (`Tools::getAdminTokenLite`). Los AJAX se hacen vía `controller=...&token=...`.

6.2 Whitelist de scripts

`Ecom_Serveraudit_Runner::$whitelist` define **qué scripts y qué argumentos** puede ejecutar el módulo. Cualquier otra cosa devuelve "Script no permitido".

Por seguridad, los scripts destructivos (`fix --apply` , `tune --restart` , etc.) **NUNCA se ejecutan desde el módulo**. Sólo se muestran los comandos SSH a copiar y pegar.

6.3 escapeshellarg / escapeshellcmd

Todos los argumentos pasados a `proc_open` se filtran con regex y se escapan con `escapeshellarg()` . La ruta del script con `escapeshellcmd()` .

6.4 Lectura de logs

`Runner::readLog()` valida que la ruta esté dentro de `_PS_ROOT_DIR_` . Limita a 5MB (trunca a últimos 2MB si supera).

6.5 pSQL en queries

El módulo no hace queries personalizadas a la BD (sólo `Db::getInstance()->getVersion()` o un `SELECT VERSION()` defensivo en `AdminEcomServerauditController::getDbVersion()`). Si en futuras versiones se añaden queries: usar siempre `pSQL()` .

7. Permisos archivo

- Carpeta `sh/` : scripts con `chmod 700` (sólo dueño puede ejecutar)
- Carpeta `logs/` : `chmod 755` (sólo lectura web)
- Resto: estándar PrestaShop (`644` ficheros, `755` carpetas)

8. Compatibilidad PHP

El módulo usa **sólo funciones PHP estándar** (sin Composer): - `proc_open` (necesario para ejecutar shell) - `escapeshellarg` , `escapeshellcmd` - `disk_free_space` , `posix_geteuid` (opcional) - `file_get_contents` , `glob` - Smarty (incluido en PrestaShop)

Si `shell_exec` o `proc_open` están deshabilitadas en `disable_functions` , el módulo muestra warning en el header y todo se reduce a copy/paste SSH.

9. Multi-tienda

El módulo es de **administración pura**, no afecta al front. Los scripts shell Sí detectan multi-tienda y advierten antes de tocar BD.

10. Performance

- `audit_*.log` puede ser grande (1-5 MB típicamente).
- El parser está optimizado para no cargar más de 5MB en memoria.
- `listLogs()` usa `glob()` (rápido para <1000 ficheros).
- La pestaña Dashboard sólo parsea el último log de cada tipo.

11. Backup / restauración

Los scripts shell hacen su propio backup `.sql.gz`. El módulo NO replica el backup en su carpeta — sólo lista lo que encuentra en `/auditoria/` del PS.

12. Logs de depuración

Si `ECOM_SERVERAUDIT_DEBUG=1`, se escribe a `logs/ecom_serveraudit_YYYY-MM-DD.log`. Activable desde la pestaña Configuración del panel.

13. Cron / autoejecución

El módulo **NO crea tareas cron**. Para automatizar, usa cron del sistema con los scripts shell (ver INFO.txt en `sh/`).

14. Errores comunes

Mensaje	Causa	Solución
"shell_exec deshabilitada"	<code>disable_functions</code> la prohíbe	Usar SSH directamente
"Script no disponible"	<code>.sh</code> no existe o no es ejecutable	"Aplicar permisos a los <code>.sh</code> " en Settings
"Script no permitido"	Acción fuera de whitelist	(esperado en uso normal)
"log no accesible"	Path fuera de PS_ROOT	(esperado, seguridad)
Charts no se dibujan	<code>chart.umd.min.js</code> es placeholder	Sustituir por Chart.js 4.x oficial

Ecom Server Audit - Guía para desarrolladores

Versión: 1.0.0 **Autor:** Ecom Experts ecomyseo@gmail.com **Licencia:** AFL-3.0

1. Convenciones de código (CLAUDE.md - obligatorias)

- Cabecera **en TODOS los archivos** PHP/JS/CSS/TPL.
- **0 líneas en blanco** antes de `<?php`.
- `if (!defined('_PS_VERSION_')) exit;` tras la cabecera en cada PHP.
- `index.php` con `header('Location: ../');` `exit;` en cada subcarpeta.
- Nombre clase principal: `Ecom_Serveraudit` (coincide con `ecom_serveraudit/`).
- **SIN namespace** en el archivo principal (global namespace).
- **Carga manual** con `require_once`. **NO Composer**.
- `pSQL()` obligatorio en queries manuales (no aplicable aquí, no hay queries propias).
- `$this->context` en clase módulo. En servicios externos: `\Context::getContext()`.
- `isUsingNewTranslationSystem()` retorna `true`.
- Para futuras updates: usar `addnewfeatures()` (no ficheros `upgrade/upgrade-x.y.php`).

2. Añadir un nuevo script al toolkit

1. Crear `sh/nuevo.sh` siguiendo el patrón de los existentes (detección creds, dry-run, etc.).
2. Añadir a la whitelist en `Ecom_Serveraudit_Runner::$whitelist`: `php protected $whitelist = [ // ... 'nuevo' => ['--yes'], // args permitidos por defecto ];`
3. Si hace falta, añadir método `parse...` en `Ecom_Serveraudit_Parser`.
4. Añadir tarjeta en `views/templates/admin/_partials/tools.tpl`.
5. (Opcional) Añadir paso en `wizard.tpl`.

3. Añadir una pestaña nueva

1. Editar array `$allowed` en `AdminEcomServerauditController::initContent()`.
2. Añadir enlace en `views/templates/admin/main.tpl` dentro de `.ecsa-tabs`.
3. Crear `views/templates/admin/_partials/<nueva>.tpl`.
4. Añadir `{elseif $tab == 'nueva'}` en `main.tpl`.

4. Añadir parseo de un nuevo tipo de log

```
public static function parseMyLog($content)
{
    $data = [];
    if (preg_match('/Patrón:\s*(.+)$/m', $content, $m)) {
        $data['key'] = trim($m[1]);
    }
    return $data;
}
```

Y en el `AdminEcomServerauditAjaxController` agregar:

```
protected function ajaxParseMyLog() {
    $path = (string) Tools::getValue('path');
    $runner = new Ecom_Serveraudit_Runner();
    $content = $runner->readLog($path);
    if ($content === null) {
        $this->json(['ok' => false, 'error' => 'log no accesible']);
    }
    $this->json(['ok' => true, 'data' => Ecom_Serveraudit_Parser::parseMyLog($content)]);
}
```

Y añadirlo al array `$allowed` en `init()`.

5. Añadir un nuevo gráfico

1. En `dashboard.tpl` añada `<canvas id="ecsa-chart-X"></canvas>` y un `<script type="application/json" id="ecsa-data-X">` con los datos.
2. En `admin.js` (al final de `DOMContentLoaded`): `js renderChart('ecsa-chart-X', 'ecsa-data-X', 'bar', 'Mi label', 'rgba(...))');`

6. Variables CSS personalizables

Todo en `:root` al inicio de `admin.css`. Cambia colores ahí (paleta inspirada en Mantine/PrestaShop):

```
--ecsa-primary: #4c6ef5;
--ecsa-accent: #7950f2;
--ecsa-ok: #12b886;
--ecsa-warn: #fab005;
--ecsa-danger: #fa5252;
```

7. AJAX desde el frontend del módulo

```
function ajax(action, params, cb) { /* helper en admin.js */ }
ajax('run', { script: 'audit', args: ['--yes'] }, function(res) {
    if (res.ok) console.log(res.output);
});
```

Acciones disponibles: `run`, `readLog`, `parseAudit`, `parsePerf`, `parseSlow`, `listLogs`, `systemInfo`.

8. Permisos de los .sh

El módulo intenta `chmod 700` automáticamente. Si falla (FTP user != web user): - Configuración → "Aplicar permisos a los .sh" - O por SSH: `chmod 700 modules/ecom_serveraudit/sh/*.sh`

9. Testing local

```
# Símbolo simbólico al módulo en una PS de pruebas
ln -s /ruta/al/repo/ecom_serveraudit /var/www/html/modules/ecom_serveraudit
# Instalar desde BO > Módulos > Subir módulo > "instalar"
```

10. Estructura PSR (no aplicable)

PrestaShop no usa PSR-4 para módulos legacy. Se carga con `require_once`.

11. Compatibilidad PHP 7.4 ↔ 8.x

Código preparado para ambas: - No usa `match`, `enums`, `nullsafe ?->`, `named args` (sólo PHP 8+). - Sí usa `null coalescing ??` (PHP 7+) y `arrow fn` cuando aplica. - `is_object($controller) ? get_class($controller) : ''` (NO `$controller::class`).

12. Smarty templates

Usa siempre escape:

```
{$var|escape:'html':'UTF-8'}
{$var|escape:'javascript':'UTF-8'}
```

PROHIBIDO `nofilter`.

13. Linting

- PHPStan level 5 recomendado
- Para detectar `eval` /funciones peligrosas: regla custom

14. Versionado

Cada cambio importante: 1. Bump `$this->version` en `ecom_serveraudit.php` 2. Bump `<version>` en `config.xml` 3. Si hay cambios BD/estructura, usar `addnewfeatures()` (NO upgrade scripts) 4. Actualizar los 3 .md de `docs/`